

Assignment 2 - Extended Kalman filter and Unscented Kalman filter

- **Topic:** Designing an EKF and UKF to estimate the position using magnetic field information
- **Assessment:** The assignment will go through a pass/fail check.
- **Deadline:** 06-03-2026, 17:00
- **Submitting:** **SUBMIT ONLY** `assignment2_groupNumber.ipynb` **TO BRIGHTSPACE**, where groupNumber is your groupnumber

Instructions

Installation: The implementation is tested with python 3.9.12 and packages including

- numpy version: 1.26.4
- scipy version: 1.13.0
- matplotlib version: 3.5.1

Other (not too old or new) versions will probably also work. For windows user check the announcement ("**Fix of Issues Assignment 1**") on brightspace which could also be helpful for this assignment.

You may not use other packages for algorithm-related calculations. You only need to complete (and submit) this file. Please do not change the additional files `GP.py`, `helper.py` and `linAlg.py` and the dictionary `modelParameters` as this might result in breaking the assignment.

AI Related Policy

We strongly discourage you to use AI tools for implementation assistance. It is your understanding of the problem that is tested in the final exam.

Information

Please fill in your group number, names and student numbers in the cell below.

```
In [1]: ''' YOUR ANSWER HERE '''  
groupNumber = 6
```

```
STUDENT_1_NAME = "Giuliano Moroni"
STUDENT_1_STUDENT_NUMBER = "5680204"

STUDENT_2_NAME = "Ian Michel"
STUDENT_2_STUDENT_NUMBER = "6545475"
```

Objectives

Goal

The goal of this assignment is to estimate the position $\mathbf{x}_k$ of an agent, the same as assignment 1, but using different algorithms. The agent moves in an indoor environment and gets information about its approximate change in position $\Delta\mathbf{x}_k$ at every time-step. To model this, you may assume a linear dynamic model of the form:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_k + \mathbf{w}_k$$

Where:

- $\mathbf{x}_k$ is a 3-dimensional position vector of the agent at time-step k .
- $\Delta\mathbf{x}_k$ is the change in position.
- $\mathbf{w}_k \sim \mathcal{N}(0_{3 \times 1}, \mathbf{Q})$ is Gaussian white noise.
- $\mathbf{Q} \in \mathbb{R}^{3 \times 3}$ is the covariance of the process noise.

Since the information of the change in position $\Delta\mathbf{x}_k$ contains errors, the position estimates from the dynamic model will drift. This means that over time, the agent will lose track of where it actually is. To remedy this, the agent is carrying a magnetometer, and has a map of the magnetic field. The magnetometer samples a measurement of the magnetic field $\mathbf{y}_k$ at every position $\mathbf{x}_k$. The map of the magnetic field is modeled by a non-linear function: a reduced rank Gaussian process (you do not need to understand, model or change the GP model, only to call the function evaluation). This results in the non-linear measurement model:

$$\mathbf{y}_k = f(\mathbf{x}_k) + \mathbf{v}_k$$

Where:

- $\mathbf{y}_k$ is a 3-dimensional measurement vector of the magnetic field at time-step k .
- $f(\cdot)$ is the reduced rank Gaussian process function.
- $\mathbf{v}_k \sim \mathcal{N}(0_{3 \times 1}, \mathbf{R})$ is Gaussian white noise.
- $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is the covariance of the measurement noise.

First load in the data

To start, first load in the data below All variables you will use are numpy arrays, where

- `magnetometerMeasurements` $\in \mathbb{R}^{3 \times K}$ is a numpy array containing the magnetometer measurements
- `magnetometerInitialPosition` $\in \mathbb{R}^{3 \times 1}$ is a numpy array containing the initial position of the magnetometer
- `deltaMagnetometerPositions` $\in \mathbb{R}^{3 \times K}$ is a numpy array containing the change in position of the magnetometer

Warning

- Additionally you are provided with a dictionary `modelParameters` which is used in creating magnetic field maps. DO NOT CHANGE dictionary as this might break the assignment. To create the EKF and UKF you DO NOT need this dictionary.

```
In [2]: ''' DO NOT CHANGE THE CODE BELOW '''
import numpy as np
import matplotlib.pyplot as plt

import GP as GP

import linAlg as linAlg
import helper as helper
np.random.seed(groupNumber)

(magnetometerMeasurements,
 magnetometerInitialPosition,
 deltaMagnetometerPositions,
 modelParameters) = helper.initializeKalmanFilterAssignment(groupNumber % 29)

print(modelParameters.keys())
print(modelParameters['NtimeSteps'])
```

```
dict_keys(['theta', 'Din', 'Npred', 'NtimeSteps', 'domain', 'margin', 'GPweights',
'GPdataInverse', 'numberOfBasisFunctions', 'posGroundTruth', 'Nm', 'NmLinCf', 'indices', 'Lambda', 'lambdaLinCf'])
712
```

Same as the first assignment of particle filter, below, a function

`helper.makeDeadReckoningPlots()` is given to plot the map of the magnetic field and the ground truth position of the path taken by the agent. The agent does not know this path, it only has access to its initial position, the change in position at every time-step and the magnetometer measurements. You can reuse the code for the plots.

Your first task it to use the agent's dynamic model to try and reconstruct the ground truth path. This is also known as dead reckoning and should be done using data stored in the variables `magnetometerInitialPosition` and `deltaMagnetometerPositions`. Save the agent's positions according to the dynamic model in the variable `deadReckoning` $\in \mathbb{R}^{3 \times K}$. Additionally, compare the magnetometer measurements with the map of the

magnetic field. To achieve this, store the norm of the magnetic field at every time-step in the variable `magneticFieldNorm` $\in \mathbb{R}^K$.

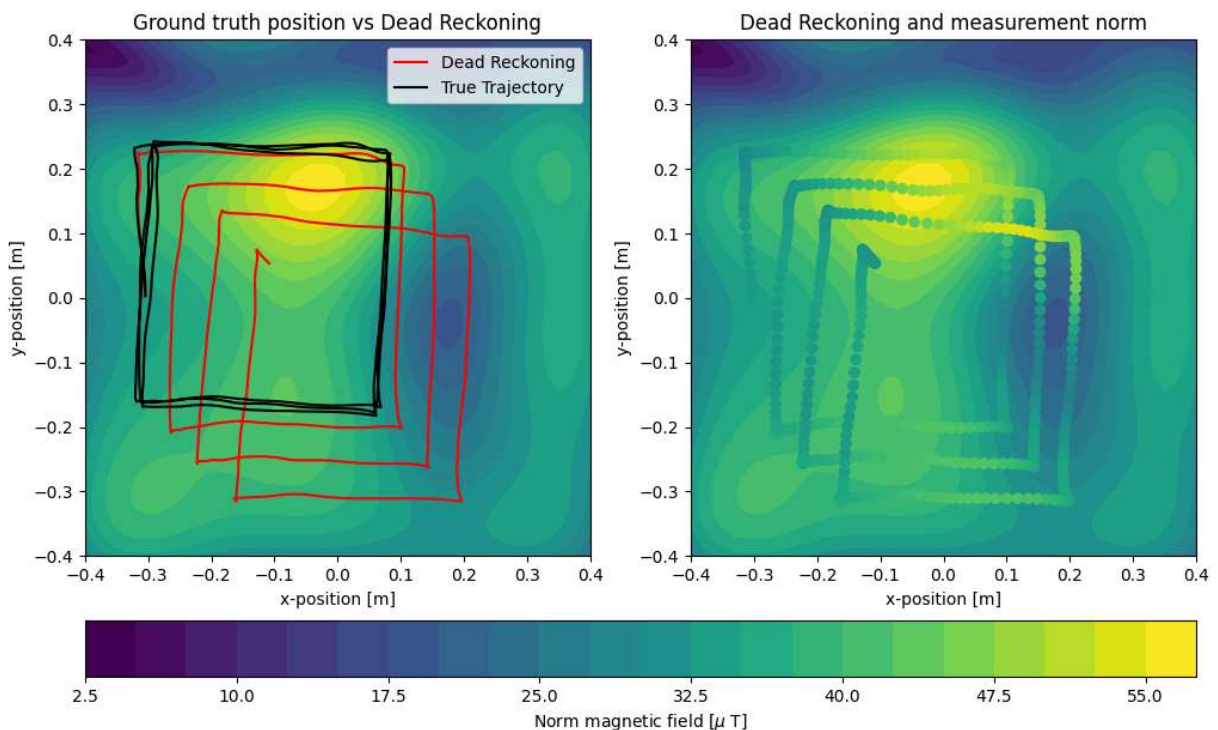
This function should output two subfigures. Every subfigure includes the magnetic field map in the background. On the left the ground truth positions are plotted together with the dead reckoning positions. On the right, the magnetic field measurements are plotted with the ground truth positions over the magnetic field map.

```
In [3]: p = magnetometerInitialPosition.copy()
delta = deltaMagnetometerPositions.copy()
N = deltaMagnetometerPositions.shape[1]

deadReckoning = np.zeros((3,N))
for i in range(deltaMagnetometerPositions.shape[1]):
    p = (p + delta[:, i:i+1])
    deadReckoning[:,[i]] = p

magneticFieldNorm = (np.linalg.norm(magnetometerMeasurements,axis=0))

helper.makeDeadReckoningPlots(deadReckoning, magneticFieldNorm, modelParameters)
```



EKF and UKF

To use magnetometer measurements to reduce the positional drift observed in the figure above, besides PF in the previous lectures, you can also use EKF and UKF.

EKF Implementation

You start from implementing EKF. Your task is to fill in the following functions.

Task 1

Fill in the `dynamicUpdateEKF()` function according to the dynamic model given above.

Note: You do not have to change the inputs and outputs to the given functions.

[illegible]

Task 2

Complete the function `measurementUpdateEKF()` according to the measurement model given above.

To implement the measurement model, you will need to make predictions of the magnetic field. You DO NOT need to create this function yourself. To make predictions, use the function `GP.makeMagneticFieldPrediction(positions, modelParameters)` function make sure the input `positions` $\in \mathbb{R}^{3 \times 1}$. The output should be a matrix $\in \mathbb{R}^{3 \times 1}$ magnetic field predictions for every input location. Similarly, to find the Jacobian, you must use the `GP.makeMagneticFieldJacobian(position, modelParameters)` function make sure the input position is a vector of 3×1 . The output should be a matrix of 3×3 .

```
In [5]: def measurementUpdateEKF(position,
                                     magnetometerMeasurement,
                                     posCov,
                                     measurementCovarianceEKF,
                                     modelParameters):

    ''' Measurement update '''

    magneticFieldPrediction = GP.makeMagneticFieldPrediction(position, modelParameters)
    jacobianMagneticFieldPrediction = GP.makeMagneticFieldJacobian(position, modelParameters)

    inv4K = np.linalg.inv(jacobianMagneticFieldPrediction @ posCov @ (jacobianMagneticFieldPrediction.T + measurementCovarianceEKF))

    K = posCov @ (jacobianMagneticFieldPrediction).T @ inv4K
```

```

position = position + K @ (magnetometerMeasurement - magneticFieldPrediction)
posCov = posCov - K @ jacobianMagneticFieldPrediction @ posCov

return position, posCov, jacobianMagneticFieldPrediction

```

`extendedKalmanFilter()` is given as follows. You don't need to change it.

```

In [6]: ''' DO NOT CHANGE THE CODE BELOW '''

def extendedKalmanFilter(position,
                        deltaPosition,
                        magnetometerMeasurement,
                        posCov,
                        motionCovarianceEKF,
                        measurementCovarianceEKF,
                        modelParameters):
    ''' Estimates positions'''

    ''' Dynamic update '''
    if modelParameters['NtimeStepEKF'] > 0:
        (position,
         posCov) = dynamicUpdateEKF(position,
                                     deltaPosition,
                                     posCov,
                                     motionCovarianceEKF,
                                     modelParameters)

    ''' Measurement update '''
    (position,
     posCov, jacobianMagneticFieldPrediction) = measurementUpdateEKF(position,
                                                                        magnetometerMeasurement,
                                                                        posCov,
                                                                        measurementCovarianceEKF,
                                                                        modelParameters)

    return position, posCov, jacobianMagneticFieldPrediction

```

Task 3

Be sure to set the **initial position** and the **prior** on it correctly and a **suitable measurement covariance** as you did in assignment 1.

You could start with the following parameters

- `motionCovarianceUKF = np.eye(3)*0.001**2`
- `measurementCovarianceUKF = np.eye(3)*1.5**2`

Make sure `posCovarianceUKF` is a 3d tensor with the shape of `[3,3,NtimeSteps]`, to save the position covariances for every time step. To get better results, you might have to tune these.

```

In [7]: ''' Initialize the EKF '''
modelParameters['NtimeStepEKF'] = 0

```

```

motionCovarianceEKF = np.eye(3)*0.0035**2
measurementCovarianceEKF = np.eye(3)*3.**2

posEstEKF = np.zeros((3,712))
posEstEKF[:,0:1] = magnetometerInitialPosition
posCovarianceEKF = np.zeros((3,3,712))

# Inital position covariance:
posCovarianceEKF[:, :, 0] = np.eye(3) * np.array((0.00045, 0.00048, 0.00015))
# print(f"InitCov:\n{posCovarianceEKF[:, :, 0]}")

jacobian_history = np.zeros((3, 3, 712))

''' Run the EKF'''
for timendx in range(modelParameters['NtimeSteps']-1):
    deltaPosEKF = deltaMagnetometerPositions[:, timendx:timendx+1]
    ymeasEKF = magnetometerMeasurements[:, timendx:timendx+1]
    (posEstEKF[:, timendx+1:timendx+2],
     posCovarianceEKF[:, :, timendx+1], jacobian_history[:, :, timendx+1]) = extendEKF(
                                                deltaPosEKF,
                                                ymeasEKF,
                                                posCovarianceEKF[:, :, timendx],
                                                motionCovarianceEKF,
                                                measurementCovarianceEKF,
                                                modelParameters)

    modelParameters['NtimeStepEKF'] += 1

```

discussion

Elaborate on the effects of tuning the different variables:

- `motionCovarianceEKF`
 - `measurementCovarianceEKF`
 - The initial position covariance `posCovarianceEKF[:, :, 0]`
1. `motionCovarianceEKF` - Changing this parameter had a large effect on our predictions, after crossing a threshold of approx $\geq 0.01^2$ large distortions in the EKF predicted path (`posEstEKF`) became apparent. This was attributed to the fact that we were injecting so much uncertainty in our dynamic estimate that the subsequent measurement update from the EKF then refined the estimate poorly (based on very large Kalman gain, from a large `posCov`), most likely further stressed by the multi-modal nature of the magnetic field. Which, conflicts with our Gaussian assumptions in the EKF. Besides this the variable increases the amount of covariance in dynamic estimates, shown by the blue covariance circles expanding as we increase `motionCovarianceEKF` .
 2. `measurementCovarianceEKF` - This parameter also had very notable effects on our estimated position and covariance. It was noted that as we increase the `measurementCovarianceEKF` , increasing our uncertainty in our measurement model,

that the `posEstEKF` began to mirror `deadReckoning` trajectory. Intuitively this make sense, as our uncertainty is so high in the measurement update that the EKF begins to discard the influence of measurements of the position estimate. Conversely, when arbitrarily decreasing the covariance, the `posEstEKF` path became unstable, this again due to the multi-modality of the data. If we believe too singularly in our measurement, which infact can be represented by multiple different locations on the magnetic field, then our estimated path will become unstable (as seen). Therefore, a balance must be struck as it relates to the `measurementCovarianceEKF` to trust our dynamic model and reinforce it with our magnetometer data.

3. `posCovarianceEKF[:, :, 0]` - It was noted that as long as the previous two parameters were correctly tuned, the initial position covariance had a minor effect on the prediction performance. The EKF was able to quickly minimize the variance in the first tens of seconds of the simulation. The only adverse effects it introduced is in our ability to analyze the system, given then our initial covariance is very high (relative to the `motionCovarianceEKF` and `measurementCovarianceEKF`), the variations of covariance become untractable. As such, the initial covaraiance was selected to mirror the $cov(x)$, $cov(y)$, and $cov(z)$ peaks after tuning the `motionCovarianceEKF` and `measurementCovarianceEKF`, via a visual process of attempting the encapsulate the true path in our uncertain prediction path (encompass true path in blue covariance circles).

Visualize the output

```
In [8]: ''' DO NOT CHANGE THE CODE BELOW '''
helper.makeExtendedKalmanFilterPlots(deadReckoning, posEstEKF, posCovarianceEKF, mo

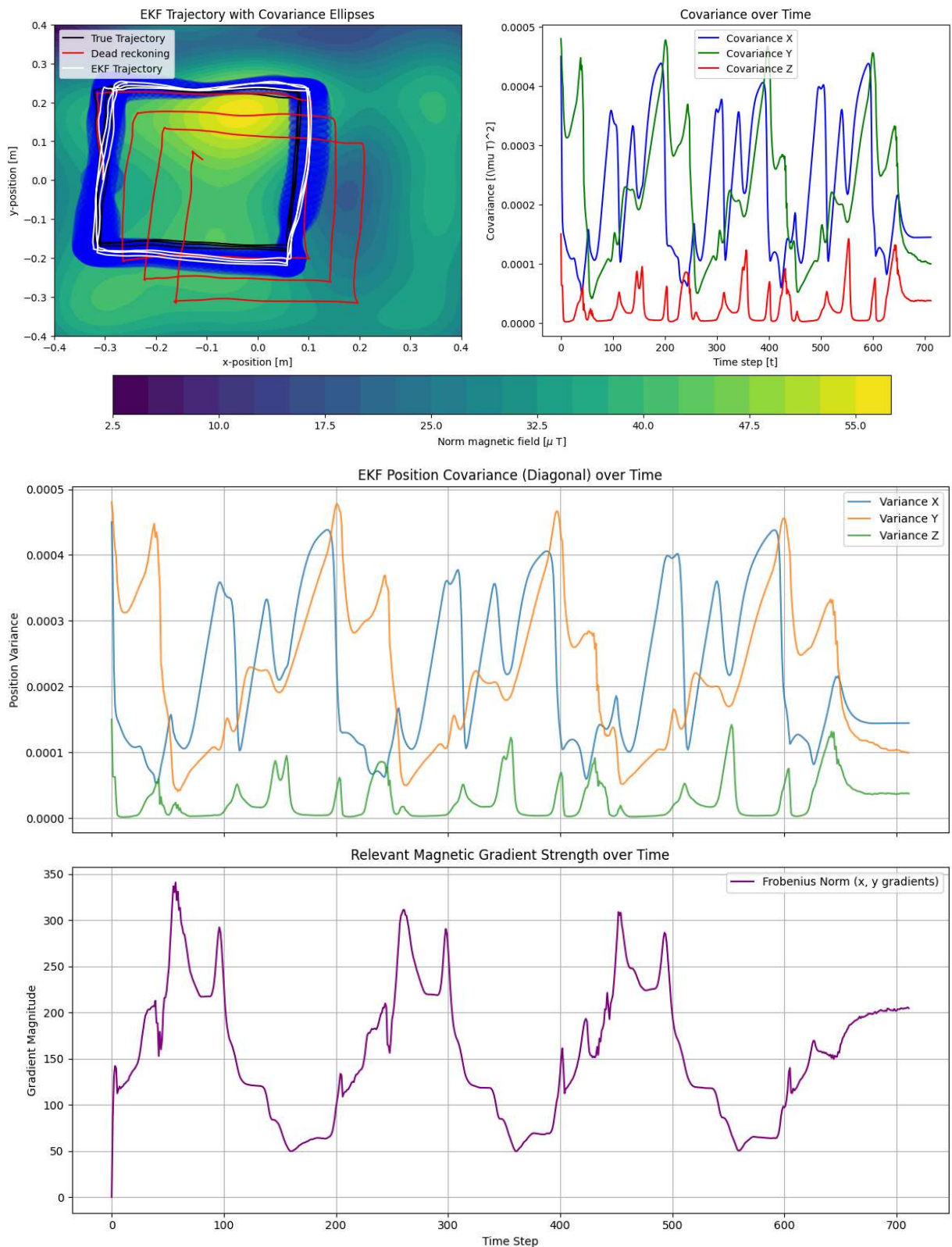
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)
time_steps = np.arange(magnetometerMeasurements.shape[1])
# print(time_steps)
# 2. Position Variance
ax1.plot(time_steps, posCovarianceEKF[0, 0, :], label='Variance X', alpha=0.8)
ax1.plot(time_steps, posCovarianceEKF[1, 1, :], label='Variance Y', alpha=0.8)
ax1.plot(time_steps, posCovarianceEKF[2, 2, :], label='Variance Z', alpha=0.8)
ax1.set_ylabel('Position Variance')
ax1.set_title('EKF Position Covariance (Diagonal) over Time')
ax1.legend()
ax1.grid(True)

# 3. Jacobian Norm (x and y gradients only)
# Slice the first two columns (derivatives w.r.t x and y) and compute Frobenius nor
# jacobian_history shape: (3, 3, K) -> slice shape: (3, 2, K)
xy_jacobian = jacobian_history[:, :2, :]
jacobian_norm_xy = np.sqrt(np.sum(xy_jacobian**2, axis=(0, 1)))

ax2.plot(time_steps, jacobian_norm_xy, label='Frobenius Norm (x, y gradients)', col
ax2.set_xlabel('Time Step')
ax2.set_ylabel('Gradient Magnitude')
```

```
ax2.set_title('Relevant Magnetic Gradient Strength over Time')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()
```



Discussion

Explain why the covariance changes over time the way it does. You can create more plots if necessary for your analysis.

Above the following extra plots are included: the (x, y, z) covariance, and the frobenius norm of the jacobian (w.r.t to the x, y gradients), over a shared time scale. From here it is clearly seen that the $cov(x)$ and $cov(y)$ increases in regions where there is less magnetic field strength. This is a result of the "richness" of data in a specific area; when the jacobian norm is low, there is less dynamic information for the measurement update, and the EKF relies on the more heavily on the dynamic model, accumulating process uncertainty over time. When the jacobian norm is high, the agent is passing through an area with strong, and distinct magnetic gradients, as such the EKF received high-quality localization data and updates more confidently.

UKF implement

Task 1

Fill in the `dynamicUpdateUKF()` function according to the dynamic model given above.

Note:

You do not have to change the inputs and outputs to the given functions.

```
In [9]: def dynamicUpdateUKF(position,
                             deltaPos,
                             posCov,
                             motionCovarianceUKF,
                             modelParameters):
    ''' Dynamic update '''
    position = position + deltaPos

    posCov = posCov + motionCovarianceUKF

    return position, posCov
```

Task 2

- Complete creating sigma points `createSigmaPoints()` and calculating sigma weights `calculateSigmaWeights()`.
- Fill in the `measurementUpdateUKF()` according to the measurement model given above.
- You can play around with UT1 and UT2, can you see any obvious differences?

```
In [149... def createSigmaPoints(stateVector,
                       stateCovariance,
                       lamdaWeights,
                       modelParameters):
    ''' Create sigma points '''
```



```

n = 3
sigmaPoints = np.zeros((n,2*n+1))
U, S, V = np.linalg.svd(stateCovariance)
rP = U @ np.diag(np.sqrt(S))
sigmaPoints[:, [0]] = stateVector
for i in range(n):
    sigmaPoints[:, [i+1]] = stateVector + np.sqrt(n+lamdaWeights)*rP[:, [i]]
    sigmaPoints[:, [i+1+n]] = stateVector - np.sqrt(n+lamdaWeights)*rP[:, [i]]

return sigmaPoints

def calculateSigmaWeights(UTnumber = 2):
    ''' Calculate the weights '''
    n = 3
    if UTnumber == 1:
        '''UT1'''
        lamdaWeights = 3-n
        weightsMean1 = lamdaWeights/(n + lamdaWeights)
        weightsMeanRest = 1/(2*(n+lamdaWeights))
        weightsCov1 = lamdaWeights/(n + lamdaWeights)
        weightsCovRest = 1/(2*(n+lamdaWeights))
        weightsMean = np.zeros(2*n+1)
        weightsMean[0] = weightsMean1
        weightsCov = np.zeros(2*n+1)
        weightsCov[0] = weightsCov1
        for i in range(weightsCov.shape[0]-1):
            weightsMean[i+1] = weightsMeanRest
            weightsCov[i+1] = weightsCovRest

    else:
        '''UT2'''
        lamdaWeights = 10**(-6)
        weightsMean1 = lamdaWeights/(n + lamdaWeights)
        weightsMeanRest = 1/(2*(n+lamdaWeights))
        weightsCov1 = lamdaWeights/(n + lamdaWeights) + 3- 10**(-6)
        weightsCovRest = 1/(2*(n+lamdaWeights))
        weightsMean = np.zeros(2*n+1)
        weightsMean[0] = weightsMean1
        weightsCov = np.zeros(2*n+1)
        weightsCov[0] = weightsCov1
        for i in range(weightsCov.shape[0]-1):
            weightsMean[i+1] = weightsMeanRest
            weightsCov[i+1] = weightsCovRest

    return lamdaWeights, weightsMean, weightsCov

def measurementUpdateUKF(position,
                           magnetometerMeasurement,
                           posCovariance,
                           measurementCovarianceUKF,
                           lamdaWeights,
                           weightsMean,
                           weightsCovariance,

```

```

        modelParameters):

    ''' Measurement update '''
    n = 3
    sigmaPoints = createSigmaPoints(position, posCovariance, lamdaWeights, modelPar

    ySigma = GP.makeMagneticFieldPrediction(sigmaPoints, modelParameters)

    yEst = np.zeros((3,1))
    Ck = np.zeros((3,3))
    Sk = np.zeros((3,3))

    for i in range(2*n + 1):
        yEst += weightsMean[i]*ySigma[:,[i]]
        Ck += weightsCovariance[i] * (sigmaPoints[:,[i]] - position) @ (ySigma[:,[i]]
        Sk += weightsCovariance[i] * (ySigma[:,[i]] - yEst) @ (ySigma[:,[i]] - yEs

    Sk += measurementCovarianceUKF
    Kk = Ck @ np.linalg.inv(Sk)
    position = position + Kk @ (magnetometerMeasurement - yEst)
    posCovariance = posCovariance - Kk@Sk@Kk.T

    return position, posCovariance

```

`unscentedKalmanFilter()` is given as follows. You don't need to change it.

In [150...

```

''' DO NOT CHANGE THE CODE BELOW '''
def unscentedKalmanFilter(position,
                          deltaPos,
                          magnetometerMeasurement,
                          posCovariance,
                          motionCovarianceUKF,
                          measurementCovarianceUKF,
                          lamdaWeights,
                          weightsMean,
                          weightsCovariance,
                          modelParameters):

    if modelParameters['NtimeStepUKF'] > 0:
        ''' State update '''
        position, posCovariance = dynamicUpdateUKF(position,
                                                    deltaPos,
                                                    posCovariance,
                                                    motionCovarianceUKF,
                                                    modelParameters)

        ''' Measurement update '''
        position, posCovariance = measurementUpdateUKF(position,
                                                        magnetometerMeasurement,
                                                        posCovariance,
                                                        measurementCovarianceUKF,
                                                        lamdaWeights,
                                                        weightsMean,
                                                        weightsCovariance,
                                                        modelParameters)

```

```
return position, posCovariance
```

Task 3

Be sure to set the **initial position** and the **prior** on it correctly and a **suitable measurement covariance** as you did in assignment 1.

You could start with the following parameters

- `motionCovarianceUKF = np.eye(3)*0.001**2`
- `measurementCovarianceUKF = np.eye(3)*1.5**2`

Make sure `posCovarianceUKF` is a 3d tensor with the shape of `[3,3,NtimeSteps]`, to save the position covariances for every time step. To get better results, you might have to tune these.

In [175...

```
''' Initialize the UKF '''
modelParameters['NtimeStepUKF'] = 0

motionCovarianceUKF = np.eye(3)*0.000005
measurementCovarianceUKF = np.eye(3)*10
posEstUKF = np.zeros((3,712))
posEstUKF[:,0:1] = magnetometerInitialPosition
posCovarianceUKF = np.zeros((3,3,modelParameters['NtimeSteps']))
posCovarianceUKF[:, :, 0] = np.eye(3)*0.00000000000001

lamdaWeights, weightsMean, weightsCovariance = calculateSigmaWeights()
''' Run the UKF'''
for timendx in range(modelParameters['NtimeSteps']-1):
    (posEstUKF[:, timendx+1:timendx+2],
     posCovarianceUKF[:, :, timendx+1]) = unscentedKalmanFilter(posEstUKF[:, timendx],
                                                                deltaMagnetometerPositions[:, timendx],
                                                                magnetometerMeasurements[:, timendx],
                                                                posCovarianceUKF[:, :, timendx],
                                                                motionCovarianceUKF,
                                                                measurementCovarianceUKF,
                                                                lamdaWeights,
                                                                weightsMean,
                                                                weightsCovariance,
                                                                modelParameters)

    modelParameters['NtimeStepUKF'] += 1

posCovarianceEKF = np.zeros((3, 3, modelParameters['NtimeSteps']))
posCovarianceEKF[:, :, 0] = np.eye(3)*0
```

discussion

Elaborate on the effects of tuning the different variables:

- `motionCovarianceUKF`
- `measurementCovarianceUKF`

- The initial position covariance `posCovarianceUKF[:, :, 0]`

Is it different from tuning the variables for the EKF?

For the UT1, tuning these variables has to be very precise; typical tuning variations (magnitude 10 changes) have very big effects on how well the model works. We had to tune it perfectly in order to make the UKF work well in estimating the position of the magnetometer. This is true for all three of the covariances we can tune. For UK2, there is slightly less influence of the tuning of the covariances; the motion covariance and measurement covariance particularly have less influence. This may be as the range of the sigma points is smaller, meaning that they are more "stable", as in small variations impact them less than for sigma points far away from the central sigma point (prior estimate). The initial position covariance however has to be kept under a certain bound, or else the filter will not work. This is quite different to the EKF, where you can tweak the covariances with a bigger degree of freedom.

Motion Covariance: Increasing this beyond a certain threshold leads to the filter breaking down. Specifically, we see that the effects only hit after a certain amount of time passes, showing that the filter is no longer able to compensate for a high uncertainty after longer periods.

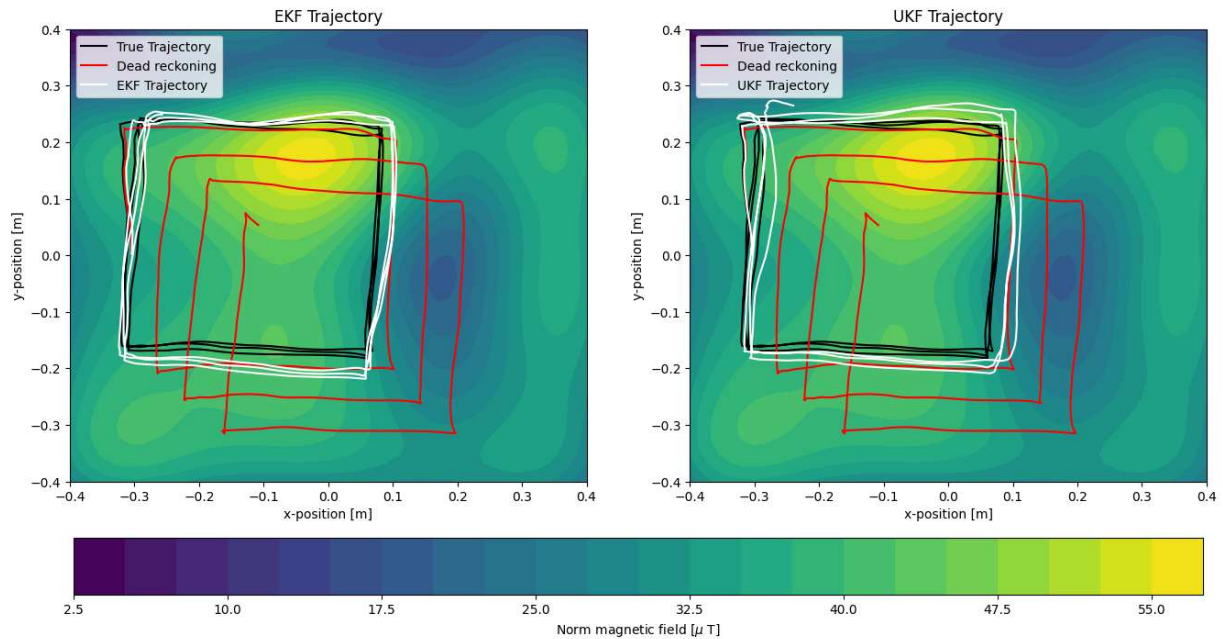
Measurement Covariance: In a similar way to the EKF, when the measurement covariance is too high, the filter recreates the "dead reckoning" movement. This is because the model no longer uses the estimates (the measurement covariance goes into S_k , which decreases the Kalman gain, and decreases how much the estimate is used).

Initial position covariance: it was found that increasing it beyond a certain point ruins the filter (as already stated). However, if it was decreased to be close to 0, the filter also replicated the movement of the dead reckoning estimate.

compare EKF and UKF

Visualize the output

```
In [176... helper.makeKalmanFilterPlots(deadReckoning, posEstEKF, posEstUKF, modelParameters)
```



Discussion

Elaborate on the two plots.

- Compare the results from the UKF with the EKF? Does one provide better results? Elaborate!
- Comment on the reason for choosing either EKF, UKF or PF for this non-linear estimation problem. Also relate this to the results from assignment 1.
- In what type of estimation problems would you expect to see very different results for the EKF, UKF and PF?

1. Both the UKF and the EKF have satisfactory (if not good!) results. However there are small differences between the two. First of all, the EKF is much more consistent with past cycles (when the magnetometer arrives to the same spot); the UKF has more variance in each cycle. However, the UKF tends to have much less error in certain parts of the cycle, for example in the bottom part of the square. Overall, the results are almost equal, if not a bit better for the EKF. This may be because of the magnetic field not being highly nonlinear.
2. The EKF should be chosen for this problem; it is well suited for a not-very-nonlinear estimation problem. All of the methods provide a good results, the particle filter also having excellent results with enough particles. However, the EKF provides good results with a fraction of the time the particle filter takes (the EKF is also about 6x faster than the UKF, which might become relevant in high-dimensionality problems that require more sigma points).

Discussion

Elaborate on the two plots.

- Compare the results from the UKF with the EKF? Does one provide better results? Elaborate!
 - Comment on the reason for choosing either EKF, UKF or PF for this non-linear estimation problem. Also relate this to the results from assignment 1.
 - In what type of estimation problems would you expect to see very different results for the EKF, UKF and PF?
1. Both the UKF and the EKF have satisfactory (if not good!) results. However there are small differences between the two. First of all, the EKF is much more consistent with past cycles (when the magnetometer arrives to the same spot); the UKF has more variance in each cycle. However, the UKF Tends to have much less error in certain parts of the cycle, for example in the bottom part of the square. Overall, the results are almost equal, if not a bit better for the EKF. This may be because of the magnetic field not being highly nonlinear.
 2. The EKF should be chosen for this problem; it is well suited for a not-very-nonlinear estimation problem. All of the methods provide a good results, the particle filter also having excellent results with enough particles. However, the EKF provides good results with a fraction of the time the particle filter takes (the EKF is also about 6x faster than the UKF, which might become relevant in high-dimensionality problems that require more sigma points).
 3. There are two distinguishing characteristic of an arbitrary estimation problem that would result in very different results between EKF and UKF, and Particle Filter (PF). This being the how non-linear the dynamics of the model are, how gaussian the posterior used in the measurement model is and size of the state-space. EKF, and UKF use approximations of non-linear behaviour (first-order gradient, and sigma-points), and as such will poorer performance in estimation problem with more non-linearity in the state-space representation of the system. Conversely, as the PF directly samples from the state-space, and it negates the drawbacks of the EKF and UKF in these context. Similarly as it relates to how Gaussian the posterior is, PF is suited for multi-modal examples due it's exact sampling nature, whereas EKF and UKF, expect and Gaussian posterior, and perform worse in multi-modal contexts. When the state-space is excessively large, or computational resources are limited. The PF will become limited in applicaiton. As we need to sample, and resample a large number of discrete points, in n -dimensional space the computational complexity of PF scales with $\mathcal{O}(N \times n^2)$, with N being the number of particles, and $N \gg n$ typically. This compared to $\mathcal{O}(n^3)$ for EKF and UKF, PF will have to limit the number of particles in copputationally constrained contexts, resulting in less accurate estimation, comparatively although EKF and UKF might be poorer at handling non-linearities, they would still out perform a limited PF in these cases.

Assignment 1 - Particle filter

- **Topic:** Designing a particle filter to estimate the position using magnetic field information
- **Assessment:** The assignment will be graded according to the rubric found on Brightspace
- **Deadline:** 27-02-2025, 17:00
- **Submitting:** **SUBMIT ONLY** `assignment3_groupNumber.ipynb` **TO BRIGHTSPACE.** where groupNumber is your group number

Instructions

Installation: The virtual environment `venvPSF` has all the necessary packages pre-installed. Make sure to select the virtual environment when running the code. This can be done in the Jupyter notebook (in vscode) by clicking select kernel in the top right, then python environments, then myenv (python 3.11.9).

If you choose to install the virtual environment yourself, the full list of dependencies is in `requirements.txt`. This can be done using the steps as shown in `howToInstallVirtualEnvironments.txt`

You may not use other packages for algorithm-related calculations. You only need to complete (and submit) this file. Please do not change the additional files `GP.py`, `helper.py` and `linAlg.py` and the dictionary `modelParameters` as this might result in breaking the assignment.

AI Related Policy

The use of AI is not allowed.

Information

Please fill in your group number, names and student numbers in the cell below.

```
In [1]: groupNumber = 6

STUDENT_1_NAME = "Giuliano Moroni"
STUDENT_1_STUDENT_NUMBER = "5680204"
```

```
STUDENT_2_NAME = "Ian Michel"
STUDENT_2_STUDENT_NUMBER = "6545475"
```

Objectives

Goal

The goal of this assignment is to estimate the position $\mathbf{p}_t$ of an agent, for $t = 1, \dots, T$. The agent moves in an indoor environment and gets information about its approximate change in position $\Delta\mathbf{p}_t$ at every time-step. To model this, you may assume a linear dynamic model of the form:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \Delta\mathbf{p}_{t+1} + \mathbf{w}_{t+1}$$

Where:

- $\mathbf{p}_t$ is a 3-dimensional position vector of the agent at time t .
- $\Delta\mathbf{p}_t$ is the change in position.
- $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}_{3 \times 1}, \mathbf{Q})$ is Gaussian white noise.
- $\mathbf{Q} \in \mathbb{R}^{3 \times 3}$ is the covariance of the process noise.

Since the information of the change in position $\Delta\mathbf{p}_t$ contains errors, the position estimates from the dynamic model will drift. This means that over time, the agent will lose track of where it actually is. To remedy this, the agent is carrying a magnetometer, and has a map of the magnetic field. The magnetometer samples a measurement of the magnetic field $\mathbf{y}_t$ at every position $\mathbf{p}_t$. The map of the magnetic field is modeled by a non-linear function: a reduced rank Gaussian process (you do not need to understand, model or change the GP model, only to call the function evaluation). This results in the non-linear measurement model:

$$\mathbf{y}_t = f(\mathbf{p}_t) + \mathbf{v}_t$$

Where:

- $\mathbf{y}_t$ is a 3-dimensional measurement vector of the magnetic field at time-step t .
- $f(\cdot)$ is the reduced rank Gaussian process function.
- $\mathbf{v}_t \sim \mathcal{N}(\mathbf{0}_{3 \times 1}, \mathbf{R})$ is Gaussian white noise.
- $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is the covariance of the measurement noise.

First load in the data

To start, first load in the data below. All variables you will use are numpy arrays, where

- `magnetometerMeasurements` $\in \mathbb{R}^{3 \times T}$ is a numpy array containing the magnetometer measurements
- `magnetometerInitialPosition` $\in \mathbb{R}^{3 \times 1}$ is a numpy array containing the initial position of the magnetometer
- `deltaMagnetometerPositions` $\in \mathbb{R}^{3 \times T}$ is a numpy array containing the change in position of the magnetometer

Warning

- Additionally you are provided with a dictionary `modelParameters` which is used in creating magnetic field maps. DO NOT CHANGE dictionary as this might break the assignment. To create the particle filter you DO NOT need this dictionary.

```
In [2]: ''' DO NOT CHANGE THE CODE BELOW '''
import numpy as np
import matplotlib.pyplot as plt

import GP as GP
import linAlg as linAlg
import helper as helper
np.random.seed(groupNumber)

''' load in the data'''
(magnetometerMeasurements,
 magnetometerInitialPosition,
 deltaMagnetometerPositions,
 modelParameters) = helper.initializeParticleFilterAssignment(groupNumber % 29)
```

Below, a function is given to plot the map of the magnetic field and the ground truth position of the path taken by the agent. The agent does not know this path, it only has access to its initial position, the change in position at every time-step and the magnetometer measurements.

Your first task it to use the agent's dynamic model to try and reconstruct the ground truth path. This is also known as dead reckoning and should be done using data stored in the variables `magnetometerInitialPosition` and `deltaMagnetometerPositions`. Save the agent's positions according to the dynamic model in the variable `deadReckoning` $\in \mathbb{R}^{3 \times T}$. Additionally, compare the magnetometer measurements with the map of the magnetic field. To achieve this, store the norm of the magnetic field at every time-step in the variable `magneticFieldNorm` $\in \mathbb{R}^T$.

This function should output three subfigures. Every subfigure includes the magnetic field map in the background. On the left the ground truth positions are plotted together with the dead reckoning positions. In the middle the magnetic field measurements are plotted with the ground truth positions on top of the magnetic field map. On the right, the magnetic field measurements are plotted with the ground truth positions over the magnetic field map.

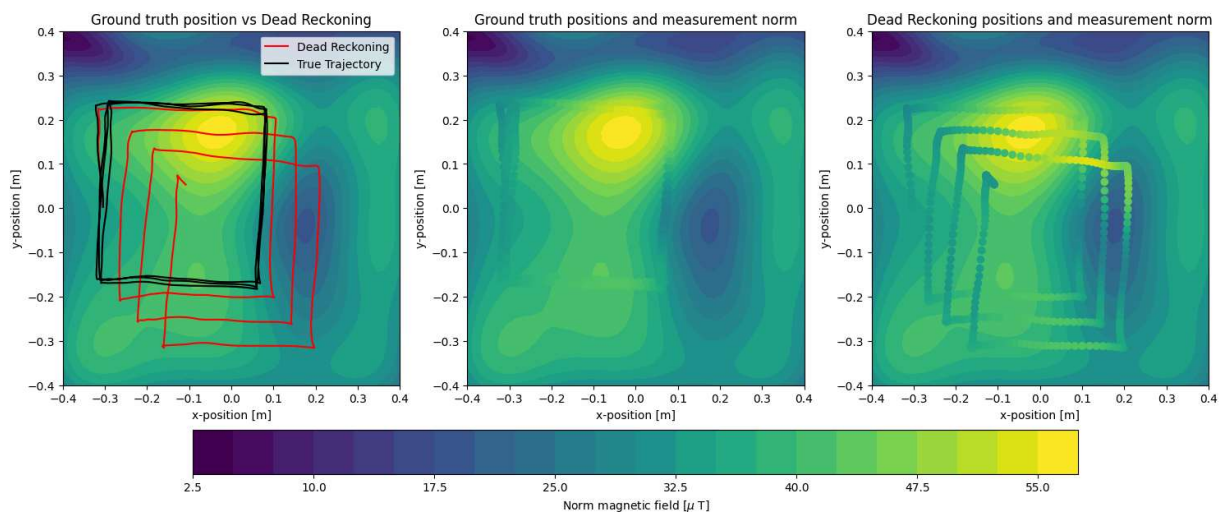
```

In [3]: p = magnetometerInitialPosition.copy()
delta = deltaMagnetometerPositions.copy()
N = deltaMagnetometerPositions.shape[1]
#print(p0)
#print(deltaMagnetometerPositions)
deadReckoning = np.zeros((3,N))
for i in range(deltaMagnetometerPositions.shape[1]):
    p = (p + delta[:, i:i+1])
    deadReckoning[:,i] = p
# print(deadReckoning.shape[1])
# print(N)

magneticFieldNorm = (np.linalg.norm(magnetometerMeasurements, axis=0))
# print(magneticFieldNorm.shape[0])
# print(magnetometerMeasurements)
# print(magneticFieldNorm)

helper.makeInitialPositionPlots(deadReckoning, magneticFieldNorm, modelParameters)

```



Discuss

Comment below on the three plots.

- Does the dynamic model provide accurate enough information about the agent's position? What will happen to the position of the agent if it moves for longer periods of time?
- Based on the magnetic field map shown in the background, explain why or why not the magnetic field is a good source of information for position estimation.
- Explain why the measurements do or do not agree with the magnetic field map (for the middle and the right plots).

1. The dynamic model alone does not provide enough information about the agents position, this is evident from the drift of our deadreckoning trajectory. The uncertainty inherent in the process noise (from the dynamic mdoel) causes error accumulation in the estimated position if not accounted for, especially over longer periods of time.

2. The magnetic field is a very good source of information; we see that here, the environment has distinct areas of lower and higher magnetic field strength. As such from our magnetometer measurements, we are able to further refine our position estimate using the magnetic field map: magnetometer readings at location (x, y) should correspond to the magnetic field strength at (x, y) . In the context of a particle filter implementation, we use the field to define the measurement likelihood $p(y_k | x_k^i)$. Conversely, if the magnetic field was uniform for the environment it would be an irrelevant source of information as we would not be able to relate magnetometer readings to distinguishable location alongside our dynamic model.
3. In the middle graph, since we are shown the magnetometer readings at the true location they are taken, they obviously agree with the field map (in fact, we can barely distinguish them from each other!). In the right plot, we see that the opposite is true; the dead reckoning measurements are clearly not aligned with the magnetic field. This is due to the aforementioned drift in the position estimates, the plot is comparing magnetometer measurements at the true location to a map value at the incorrect estimated location, causing the visual disagreement.

Particle Filter Implementation

One way to reduce the positional drift observed in the figure above, is to use magnetometer measurements. To this end, you will design a particle filter. Your task is to fill in the following three functions.

Task

Fill in the `dynamicUpdate()` function according to the dynamic model given above.

Note:

You do not have to change the inputs and outputs to the given functions.

```
In [4]: def dynamicUpdate(particlePositions,
                        deltaPosition,
                        processCovariance,
                        Nparticles):

    ''' Dynamic update '''

    for n in range(Nparticles):
        # w = np.random.normal(loc=0.0, scale=processCovariance)
        # particlePositions[n] = particlePositions[n] + deltaPosition + w
        w = np.random.multivariate_normal(np.zeros(3), processCovariance)
        particlePositions[:, n] = particlePositions[:, n] + deltaPosition + w

    return particlePositions
```

Discuss

Comment on the role of the function of the dynamicUpdate in the particle filter

--> The dynamicUpdate function handles the "propagation" of the particles as the model moves forwards: we add the position change to each particles location, and inject the stochastic process covariance noise. This causes the particles to spread out along the trajectory of the model, ready to be compared with the magnetic field and given weights in the next step.

Task

Fill in the `measurementUpdate()` according to the measurement model given above.

To implement the measurement model, you will need to make predictions of the magnetic field. You DO NOT need to create this function yourself. To make predictions, use the function `GP.makeMagneticFieldPrediction(positions, modelParameters)` function make sure the input `positions` $\in \mathbb{R}^{3 \times T}$. The output should be a matrix $\in \mathbb{R}^{3 \times T}$ magnetic field predictions for every input location.

```
In [5]: def measurementUpdate(particlePositions,
                                particleWeights,
                                magnetometerMeasurement,
                                measurementCovariance,
                                Nparticles,
                                modelParameters):

    ''' Measurement update '''
    predictionsMagneticFieldParticles = GP.makeMagneticFieldPrediction(particlePosi
w_old = particleWeights.copy()
det_R = np.linalg.det(measurementCovariance)
inv_R = np.linalg.inv(measurementCovariance)

    for n in range(Nparticles):
        p_yx = (1/np.sqrt((2*np.pi)**3 * det_R)) * np.exp(-0.5*(magnetometerMeasure
particleWeights[n] = w_old[n]*p_yx
sum_w = np.sum(particleWeights)
particleWeights = particleWeights / sum_w

    return particlePositions, particleWeights
```

Discuss

Comment on the role of the function of the measurementUpdate in the particle filter

--> The measurementUpdate function handles the updating (and normalizing) of the weights of the particles. Due to the magnetic field model, we calculate the probability of y given $f(p)$ as a gaussian with the measurement covariance. We then, for every particle, multiply this with the previous weight (bootstrap particle filtering from the lecture).

Task

Fill in the remainder of the `particleFilter()` function.

```
In [6]: def particleFilter(particlePositions,
                           deltaPosition,
                           magnetometerMeasurement,
                           processCovariance,
                           measurementCovariance,
                           Nparticles,
                           modelParameters):
    ''' Estimates position based on a known map'''

    particleWeights = np.ones(Nparticles)/Nparticles ###

    if modelParameters['NtimeStepPF'] > 0:
        ''' Dynamic update '''
        particlePositions = dynamicUpdate(particlePositions,
                                          deltaPosition,
                                          processCovariance,
                                          Nparticles)

    ''' Measurement update '''
    (particlePositions,
     particleWeights) = measurementUpdate(particlePositions,
                                          particleWeights,
                                          magnetometerMeasurement,
                                          measurementCovariance,
                                          Nparticles,
                                          modelParameters)

    meanPosition = np.average(particlePositions, weights=particleWeights, axis=1) #
    maxPositionidx = np.argmax(particleWeights) ###
    maxPosition = particlePositions[:, maxPositionidx] ###

    ''' Resampling '''
    if np.sum(particleWeights) > 1e-12:
        particleWeights /= np.sum(particleWeights)
        indices = helper.systematicResample(particleWeights)
        particlePositions = particlePositions[:, indices]
    else:
        print('Degenerate particles: All the weights are close to zero !!!')
        particleWeights /= np.sum(particleWeights)
        indices = helper.systematicResample(particleWeights)

    return particlePositions, meanPosition, maxPosition
```

Discuss

Comment on the role of the function of the systematicResample in the particle filter

--> The resampling basically increases the number of particles which have high weights. This is necessary, as less accurate particles being kept would just result in the same drift present in the dead reckoning implementation. Resampling means the particle filter stays accurate (or even gets more accurate as time goes on!).

Task 4

Choose a suitable amount of particles, an initial position covariance, a suitable motion covariance and a suitable measurement covariance.

You could start with the following parameters

- `initalPositionCovariance` = `np.eye(3)*0`
- `Nparticles` = 100
- `processCovariance` = `np.eye(3)*0.001**2`
- `measurementCovariance` = `np.eye(3)*1.5**2`

To get better results, you might have to tune these.

```

In [7]: ''' Number of particles used in the particle filter '''
Nparticles = 100

''' Initial position covariance of the position '''
initialPositionCovariance = np.eye(3)*0

''' process covariance and measurement covariance '''
processCovariance = np.eye(3)*0.001*2
measurementCovariance = np.eye(3)*1.5

''' Initialize particle positions '''
particlePositions = np.ones((3, Nparticles))*magnetometerInitialPosition

''' Create matrices to store mean and best particle position at every time-step'''
meanParticlePosition = np.zeros((3, modelParameters['NtimeSteps']))
bestParticlePosition = np.zeros((3, modelParameters['NtimeSteps']))

''' Initialize and run the Particle Filter '''
modelParameters['NtimeStepPF'] = 0
for timendx in range(modelParameters['NtimeSteps']-1):
    deltaPosPF = deltaMagnetometerPositions[:, timendx+1]
    ymeasPF = magnetometerMeasurements[:, timendx+1]
    (particlePositions,
     meanParticlePosition[:, timendx+1],
     bestParticlePosition[:, timendx+1]) = particleFilter(particlePositions,
                                                         deltaPosPF,
                                                         ymeasPF,
                                                         processCovariance,
                                                         measurementCovariance,
                                                         Nparticles,
                                                         modelParameters)

    modelParameters['NtimeStepPF'] += 1

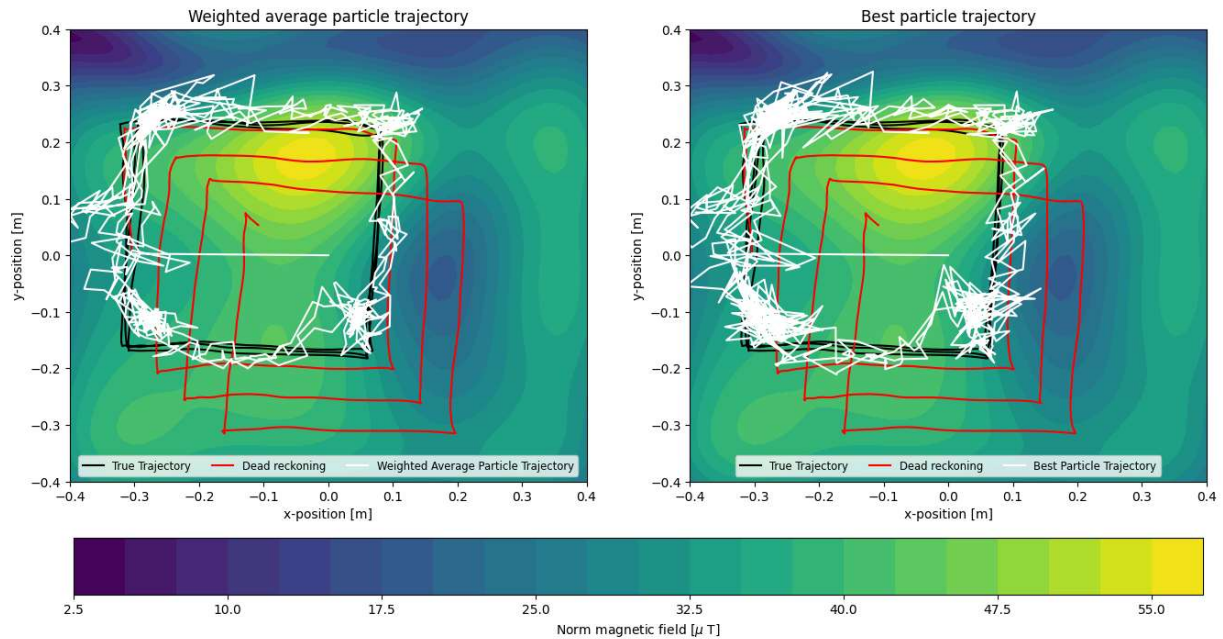
```

Visualize the output

```

In [9]: helper.makeParticleFilterPlots(deadReckoning, meanParticlePosition, bestParticlePos

```



Discuss

Elaborate on the two plots.

- Does the particle filter improve the position estimates?
 - Are the weighted average particle trajectory and the best particle trajectory the same or different? Why is that the case?
1. The particle filter (with the given parameters) greatly improves the position estimate over the dead reckoning; there is no drift, and it follows the true trajectory. However, the particle filter is by no means perfect, as we see it being quite "fuzzy", i.e. still showing a high uncertainty around the trajectory.
 2. The weighted average particle trajectory and the best particle trajectory are not exactly the same (we can see this for instance in the bottom right corner of the trajectory). However, they are very similar; this means that there is probably only one particle which has a high weight (the "best" particle), which also is the one that dominates the weighted average plot.

Task

Play around with different parameters. Comment on what happens if we choose lower or higher values for

- `initalPositionCovariance` ?
- `Nparticles` ?
- `processCovariance` ?
- `measurementCovariance` ?

And reason about why this is the case

1. Changing the initial position covariance (and keeping everything else the same) does not change the effectiveness of the particle filter much. This may be because we prescribe the initial position, and the model propagating and updating the measurements have their own mechanisms which do not depend on the initial covariance, as such the initial position covariance has little influence on the particle filter performance.
2. The number of particles greatly influences the system. If we choose a very small amount of particles (such as 10), there is a huge uncertainty in the model, and the square shape of the trajectory can barely be distinguished. Inversely, higher amount of particles (such as 500) improve the quality of the estimation (with no other parameters changed). We also see that there is no longer one particle that has a much larger weight than the others; the higher density of particles smoothes the estimate. However, more particles means more run time (10x the particles --> 10x the run time on my computer); while the number of particles should be high enough, it is costly to go too high.
3. Increasing the processCovariance has a massive effect on the output of the particle filters estimates. Increasing the processCovariance directly impacts the amount noise injected into the dynamic model, this noise additvely colors our positionDelta input, causing the dynamic model to stray from the true path. Conversely, taking a very low value and trusting our positionDelta input means that we trust the dynamic model more, in this case as deltaPostion is most likely based on the true change in position we see that the estimate converges to the true path.
4. The measurement covariance is used to model the uncertainty in our measurement; if we trust the measurement less, the measurement covariance increases, which makes the likelihood decrease (are there is a bigger "window" where the measurement could be). This does not neccessarily impact the mean plot, but it makes the best particle more jittery, as it will be less "precise" in following the measurement. The inverse is true for smaller measurement covariance

```
In [14]: ''' Number of particles used in the particle filter '''
Nparticles = 500

''' Inital position covariance of the position '''
initalPositionCovariance = np.eye(3)*0

''' process covariance and measurement covariance '''
processCovariance = np.eye(3)*0.000001*2
measurementCovariance = np.eye(3)*1.5

''' Initialize particle positions '''
particlePositions = np.ones((3, Nparticles))*magnetometerInitialPosition

''' Create matrices to store mean and best particle position at every time-step'''
```

```

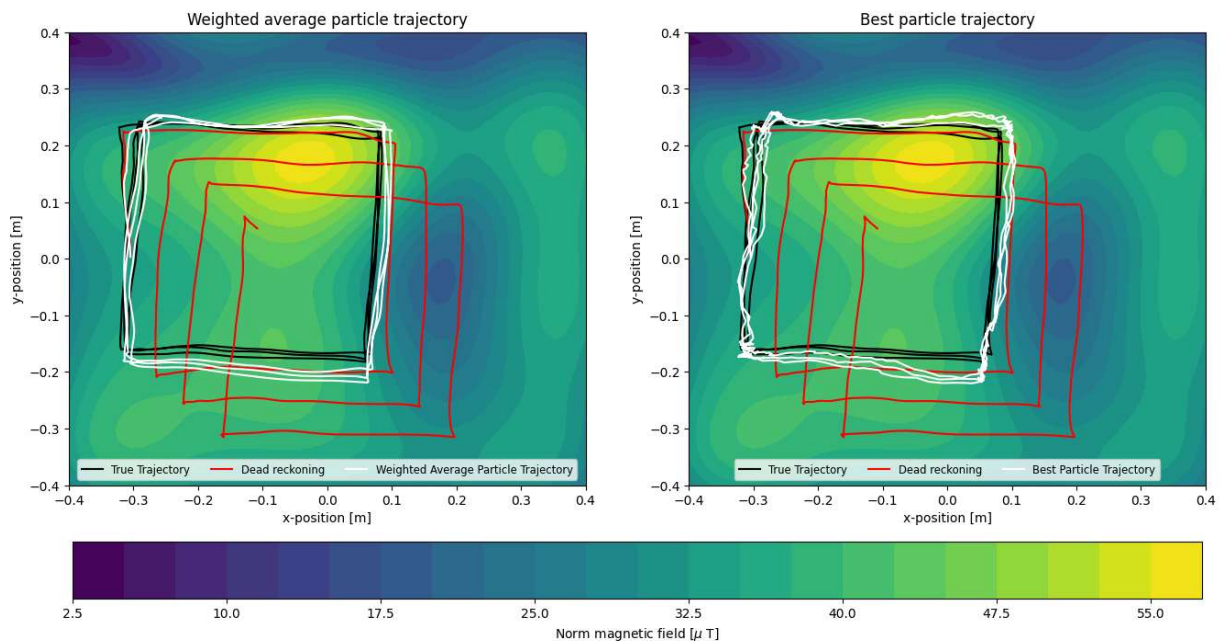
meanParticlePosition = np.zeros((3, modelParameters['NtimeSteps']))
bestParticlePosition = np.zeros((3, modelParameters['NtimeSteps']))

''' Initialize and run the Particle Filter '''
modelParameters['NtimeStepPF'] = 0
for timendx in range(modelParameters['NtimeSteps']-1):
    deltaPosPF = deltaMagnetometerPositions[:, timendx+1]
    ymeasPF = magnetometerMeasurements[:, timendx+1]
    (particlePositions,
     meanParticlePosition[:, timendx+1],
     bestParticlePosition[:, timendx+1]) = particleFilter(particlePositions,
                                                         deltaPosPF,
                                                         ymeasPF,
                                                         processCovariance,
                                                         measurementCovariance,
                                                         Nparticles,
                                                         modelParameters)

    modelParameters['NtimeStepPF'] += 1

```

In [15]: `helper.makeParticleFilterPlots(deadReckoning, meanParticlePosition[:, 1:], bestPart`



Discuss

Comment on the reason for choosing a particle filter for this estimation problem.

While, the process dynamics are linear, the measurement model, $\mathbf{y}_t = \mathbf{f}(\mathbf{p})_t + \mathbf{v}_t$, is highly non-linear. Magnetic field maps, especially of indoor environments are complex and nonuniform, and often have multiple locations with similar field strengths. Implicitly this means we can not use a Kalman Filter, for such a problem. Furthermore, using an Extended Kalman Filter, which is able to handle non-linearities, is still not ideal. It relies on the Jacobian to linearize the measurement model. Because, the magnetic map $f(p)_t$ is highly irregular,

this linearization is a poor approximation of the global nonlinearities of the magnetic map. Moreover, the EKF forces the posterior to remain a single Gaussian distribution, and in a multi-modal environment with multiple peaks (instead of one like in Gaussian) this may potentially lead to divergence. As such, while running a EKF would work, it would fail to capture the multi-modal magnetic map, potentially leading to accumulating errors. There are similar limitations when considering the Unscented Kalman Filter (UKF), which generally handles more non-linear behaviour better than the EKF, as it is still restricted to a Gaussian posterior. Conversely Particle Filter doesn't use any assumption, it handles the non-linearities of the dynamic and measurement models by directly sampling the state space (particles). This will lead to much more accurate estimates, although at a higher computational cost.